

Team Members: Caroline Locke, Chaitanya Irkar
Andrew IDs: chlocke, cirkar

1. Start by treating your cache as “always misses” and report the IPC achieved in gaussian.c. You can do this by ignoring the read_miss signal from the cache and treating it as always a miss.

IPC (Instructions Per Cycle): 0.663173

2. Set the WAYS=1 and update the POLICY to direct mapped.

a. Sweep the INDEX_BITS from 1 to 6. This corresponds to 2 to 64 sets.

b. In a table, record the counters described above as well as the IPC, read hit and miss rate.

POLICY = 0 (Direct Mapped)

IND_BITS	Total Cache Reads	Cache Read Misses	Cache Miss Rate	Cache Read Hits	Cache Hit Rate	Total Cache Writes	Cache Write Eviction	IPC
1	12643	12637	99.95	6	0.047	981	247	0.3511
2	12643	12629	99.89	14	0.111	981	245	0.3512
3	12643	12613	99.76	30	0.237	981	241	0.3514
4	12643	12581	99.51	62	0.490	981	233	0.3518
5	12643	12517	99.00	126	0.100	981	217	0.3527
6	12643	12389	97.99	254	2.009	981	185	0.3544

3. Set the POLICY to be MRU. a. Sweep all combinations of $WAYS \in \{2, 4\}$ and $INDEX_BITS \in \{2, 4, 6\}$. b. In a table, record the counters described above as well as the IPC, read hit and miss rate.

POLICY = 2 (MRU)

WAY	IND_BITS	Total Cache Reads	Cache Read Misses	Cache Miss Rate	Cache Read Hits	Cache Hit Rate	Total Cache Writes	Cache Write Evicts	IPC
2	2	12643	12620	99.82	23	0.18	981	241	0.3510
2	4	12643	12524	99.06	119	0.94	981	217	0.3523
2	6	12643	12140	96.02	503	3.98	981	121	0.3575
4	2	12643	12588	99.56	55	0.44	981	233	0.3514
4	4	12643	12396	98.05	247	1.95	981	185	0.3540
4	6	12643	11676	92.35	967	7.65	981	5	0.3642

4. Set the POLICY to be LRU. a. Sweep all combinations of $WAYS \in \{2, 4\}$ and $INDEX_BITS \in \{1, 2, 3, 4, 5, 6\}$. b. In a table, record the counters described above as well as the IPC, read hit and miss rate.

POLICY = 1 (LRU)

WAY	IND_BITS	Total Cache Reads	Cache Read Misses	Cache Miss Rate	Cache Read Hits	Cache Hit Rate	Total Cache Writes	Cache Write Evicts	IPC
2	1	12643	12629	99.88	14	0.110	981	245	0.3512
2	2	12643	12613	99.76	30	0.237	981	241	0.3514
2	3	12643	12581	99.50	62	0.490	981	233	0.3518
2	4	12643	12517	99.00	126	0.996	981	217	0.3527
2	5	12643	12389	97.99	254	2.0090	981	185	0.3544
2	6	12643	12133	95.96	510	4.033	981	121	0.3580

4	1	12643	12613	99.76	30	0.237	981	241	0.3514
4	2	12643	12581	99.50	62	0.490	981	233	0.3518
4	3	12643	12517	99.00	126	0.996	981	217	0.3527
4	4	12643	12389	97.99	254	2.009	981	185	0.3544
4	5	12643	12133	95.96	510	4.033	981	121	0.3580
4	6	12643	11671	92.31	972	7.688	981	5	0.3646

5. Sweep any other combinations of parameters you are interested in exploring and record the same counters and statistics as the other sections.

POLICY = 1, WAYS = 2, INDEX_BITS = 3

BLO CK_ SIZE	Total Cache Reads	Cache Read Misses	Cache Miss Rate	Cache Read Hits	Cache Hit Rate	Total Cache Writes	Cache Write Evicts	IPC
1	12597	12581	99.87	16	0.127	981	228	0.3518
2	12613	12581	99.74	32	0.253	981	228	0.3518
4	12643	12581	99.50	62	0.490	981	233	0.3518

6. Answer the following questions:

a. Comment on the LRU cache performance in relation to the number of WAYS and SETS. Do the same for MRU.

LRU: Keeping the index bit the same and comparing the 2-way cache and 4-way cache performances suggests that the LRU cache performance improves when increasing the number of ways. Miss rate decreases, hit rate increases, and the IPC improves. This suggests that higher associativity reduces some of the misses that occur. Keeping the ways the same and sweeping the index suggests that the LRU cache performance improves when increasing the index bits. Having more sets results in a lower number of blocks that get mapped to the same set which explains the observed higher hit rate and lower hit eviction count. Another observation is that the number of evictions decreases as the number of sets increases. While there are several noticeable improvements in the LRU cache performance, the IPC only improves marginally with changes in index bit and ways values. Hence, the IPC may show minimal improvement with LRU cache but the system may be bottlenecked elsewhere.

MRU: For the MRU, the sweep of the index bits suggests that increasing the number of sets improves the MRU performance through lower miss rate, higher hit rate, and higher IPC. Increasing the number of ways while fixing the index bits (for instance =6) improves the hit rate and IPC due to the lower number of evictions that occur. Higher set associativity is also associated with lower eviction rate. For instance when setting index bits to 6 and ways to 4 for MRU cache, we observe only 5 evictions, a very minimal amount.

b. Which replacement policy performs better? Why?

LRU. To better assess the performance of the two replacement policies, consider the case in which WAY=4 and INDEX_BITS=6. Comparing the values in the tables for the specified cache parameters, we can see that despite only a marginal difference, the LRU cache performance is slightly better than the MRU cache performance in terms of hit rate and IPC. This could be attributed to our system exhibiting temporal locality. LRU policy evicts the least recently used data while MRU policy evicts the most recently used data. Temporal locality dictates that blocks that were more recently accessed tend to be accessed more frequently. Hence, an MRU policy might evict a recently accessed cache resulting in slightly lower performance in a system that exhibits temporal locality characteristics.

c. Looking at the counters, what are the differences between WAYS=1 and SETS=32 and WAYS=2 and SETS=16 (LRU)? Why? Which of these combinations is a better use of capacity for our workload?

Both combinations have the same number of blocks. They also have similar hit rates and IPC, indicating that their performance is about the same. Hence, performance-wise both are similar. Based on our workload, the 2-way set associative cache may be more scalable, which suits our desire to optimize the system in Lab 4b.

d. From a purely IPC standpoint, what cache parametrization was best? Does this align with your expectations?

LRU WAY=4, INDEX_BITS=6. This cache parametrization has an IPC of 0.3646, which is the greatest amongst the results in the tables above. This aligns with my expectations as this configuration has max associativity and set count. This means lower miss counts which contributes to fewer stalls due to cache misses. This is a direct contributor to higher IPC.

Building off of the idea of observing hit rate normalized to size, bigger caches can see an improvement in hit rate while seeing lower normalized hit rate. This suggests that a bigger cache does automatically make it better – analysis on the optimal size balancing the tradeoff between miss rate and usage of the block is needed.